



基于Wasm构建下一代 Web服务云平台





Part 01

WASM 简介

WASM 的线性内存机制

- **概念：**线性内存是一块连续的内存，WASM指令只能读写这块内存。
- **作用：**类似于传统编程语言中的堆（heap），用于动态分配内存。
- **安全性：**WASM提供的内存访问指令确保不会超出分配的内存范围，防止越界错误。

WASM 的 Host Function

Host Function 是由宿主环境（如浏览器或服务器）提供的函数。

WASM 代码可以调用这些函数来执行特定的操作。

允许 WASM 模块与宿主环境交互，执行无法直接在 WASM 中实现的操作。

WebAssembly系统接口（WASI）介绍

WASI 概述

- **定义：** WASI 是一个为 WASM 模块提供标准化系统接口的系统层，它允许 WASM 模块在不同的宿主环境中以一致的方式访问系统资源和服务。

设计目标

- **可移植性：** 确保WASM应用在不同的操作系统和平台上保持一致的行为。
- **安全性：** 为WASM应用提供一个安全的执行环境，通过精细的权限控制限制对系统资源的访问。

作用

WASI使得WASM不仅限于浏览器，也能在服务器、边缘计算等环境中运行。



Part 02

为何选择 WASM

与现有技术对比

特性 / 技术	WASM	虚拟机 (VM)	容器	脚本语言 runtime
启动速度	极快（毫秒级）	慢（需要加载操作系统）	快（秒级）	极快（毫秒级）
调度单元	线程级/协程级	系统	进程级/系统级	线程级/协程级
执行速度	中/高(JIT)	高（略有损失）	高（native）	低/高(JIT)
拓展难度	易	难	难	易
安全性	高	高	中等	低

WASM 与协程

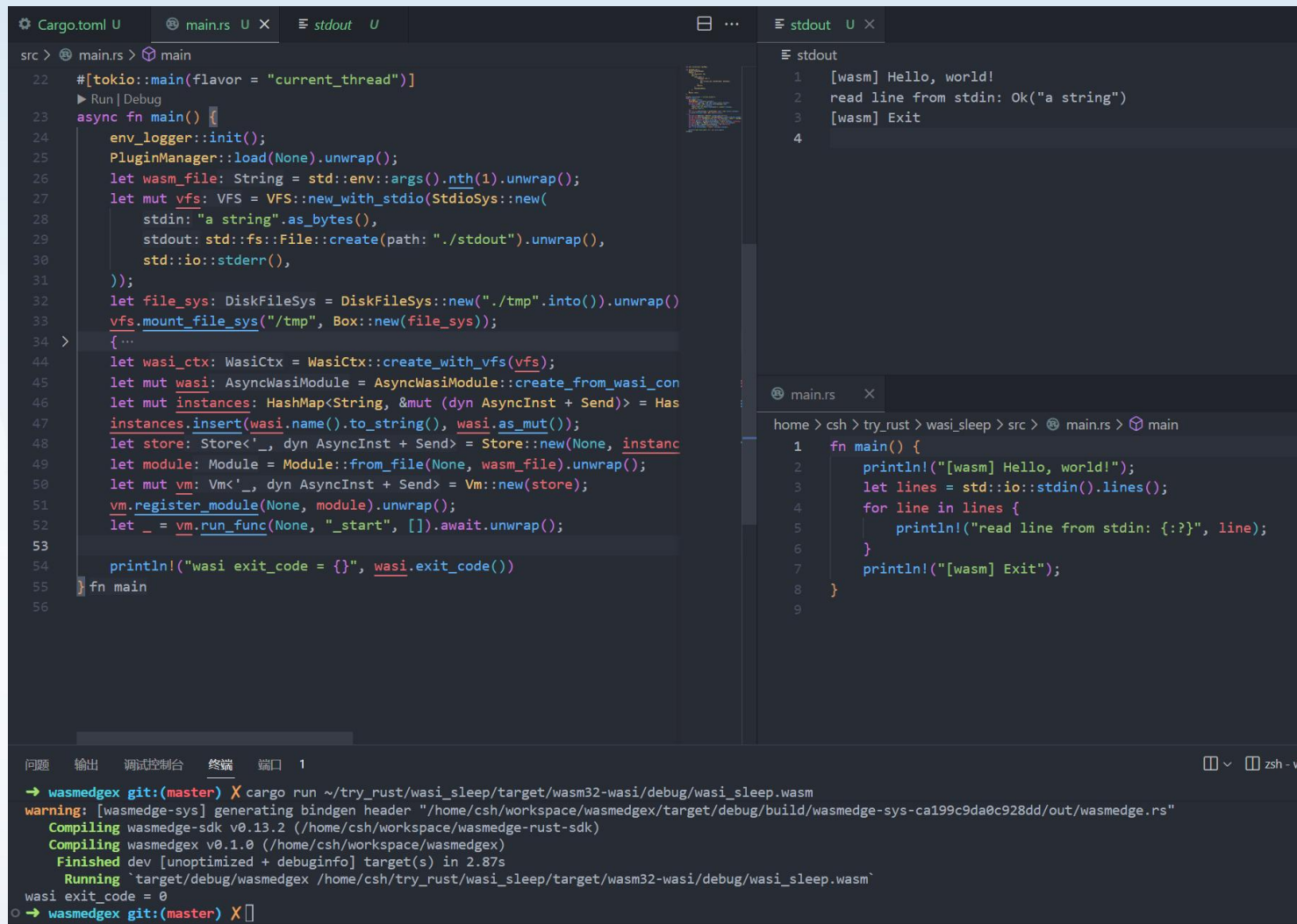
```
1 fn main() {  
2     println!("[wasm] Hello, world!");  
3     // IO  
4     std::thread::sleep(std::time::Duration::from_secs(2));  
5     println!("[wasm] Exit");  
6 }
```

```
src > main.rs > main  
22 #[tokio::main(flavor = "current_thread")]  
    ▶ Run | Debug  
23 async fn main() {  
24     tokio::spawn(async {  
25         for i: i32 in 0..20 {  
26             println!("tick {i}");  
27             tokio::time::sleep(std::time::Duration::from_millis(500)).await;  
28         }  
29     });  
30     env_logger::init();  
31     PluginManager::load(None).unwrap();  
32     let wasm_file: String = std::env::args().nth(1).unwrap();  
33 > let mut vfs: VFS = VFS::new_with_stdio(StdioSys::new( ...  
39 ));  
40 let file_sys: DiskFileSys = DiskFileSys::new("./tmp".into()).unwrap();  
41 vfs.mount_file_sys("/tmp", Box::new(file_sys));  
42 > { ...  
52 let wasi_ctx: WasiCtx = WasiCtx::create_with_vfs(vfs);  
53 let mut wasi: AsyncWasiModule = AsyncWasiModule::create_from_wasi_context(wasi_ctx).unwrap();  
54 let mut instances: HashMap<String, &mut (dyn AsyncInst + Send)> = HashMap::new();  
55 instances.insert(wasi.name().to_string(), wasi.as_mut());  
56 let store: Store<'_, dyn AsyncInst + Send> = Store::new(None, instances).unwrap();  
57 let module: Module = Module::from_file(None, wasm_file).unwrap();  
58 let mut vm: Vm<'_, dyn AsyncInst + Send> = Vm::new(store);  
59 vm.register_module(None, module).unwrap();  
60 let _ = vm.run_func(None, "_start", []).await.unwrap();  
61  
62 ⚡ println!("wasi exit_code = {}", wasi.exit_code())  
63 } fn main  
64
```

问题 1 输出 调试控制台 终端 端口 1

```
→ wasmedgex git:(master) X cargo run ~/try_rust/wasi_sleep/target/wasm32-wasi/debug/wasi_sleep.wasm  
tick 0  
tick 1  
tick 2  
tick 3  
[wasm] Exit  
wasi exit_code = 0  
→ wasmedgex git:(master) X
```


利用 Stdio 传递数据



The screenshot shows an IDE with three panels. The left panel displays the `main.rs` file with Rust code for a WASI program. The middle panel shows the `stdout` output. The bottom panel shows the terminal output of the program.

```
src > @ main.rs U X  stdout U
22 #[tokio::main(flavor = "current_thread")]
23 ▶ Run | Debug
24 async fn main() {
25     env_logger::init();
26     PluginManager::load(None).unwrap();
27     let wasm_file: String = std::env::args().nth(1).unwrap();
28     let mut vfs: VFS = VFS::new_with_stdio(StdioSys::new(
29         stdin: "a string".as_bytes(),
30         stdout: std::fs::File::create(path: "./stdout").unwrap(),
31         std::io::stderr(),
32     ));
33     let file_sys: DiskFileSys = DiskFileSys::new("./tmp".into()).unwrap()
34     vfs.mount_file_sys("/tmp", Box::new(file_sys));
35     { ...
36     let wasi_ctx: WasiCtx = WasiCtx::create_with_vfs(vfs);
37     let mut wasi: AsyncWasiModule = AsyncWasiModule::create_from_wasi_con
38     let mut instances: HashMap<String, &mut (dyn AsyncInst + Send)> = Has
39     instances.insert(wasi.name().to_string(), wasi.as_mut());
40     let store: Store<'_, dyn AsyncInst + Send> = Store::new(None, instanc
41     let module: Module = Module::from_file(None, wasm_file).unwrap();
42     let mut vm: Vm<'_, dyn AsyncInst + Send> = Vm::new(store);
43     vm.register_module(None, module).unwrap();
44     let _ = vm.run_func(None, "_start", []).await.unwrap();
45     println!("wasi exit_code = {}", wasi.exit_code())
46 }
47 fn main
```

```
stdout
1 [wasm] Hello, world!
2 read line from stdin: Ok("a string")
3 [wasm] Exit
4
```

```
main.rs X
home > csh > try_rust > wasi_sleep > src > @ main.rs > main
1 fn main() {
2     println!("[wasm] Hello, world!");
3     let lines = std::io::stdin().lines();
4     for line in lines {
5         println!("read line from stdin: {:?}", line);
6     }
7     println!("[wasm] Exit");
8 }
9
```

```
问题 输出 调试控制台 终端 端口 1
→ wasmedge git:(master) X cargo run ~/try_rust/wasi_sleep/target/wasm32-wasi/debug/wasi_sleep.wasm
warning: [wasmedge-sys] generating bindgen header "/home/csh/workspace/wasmedge/target/debug/build/wasmedge-sys-ca199c9da0c928dd/out/wasmedge.rs"
Compiling wasmedge-sdk v0.13.2 (/home/csh/workspace/wasmedge-rust-sdk)
Compiling wasmedge v0.1.0 (/home/csh/workspace/wasmedge)
Finished dev [unoptimized + debuginfo] target(s) in 2.87s
Running `target/debug/wasmedge /home/csh/try_rust/wasi_sleep/target/wasm32-wasi/debug/wasi_sleep.wasm`
wasi exit_code = 0
→ wasmedge git:(master) X
```

自定义 WASI 实现虚拟文件系统

```
src > main.rs > main
23 async fn main() {
24     env_logger::init();
25     PluginManager::load(None).unwrap();
26     let wasm_file: String = std::env::args().nth(1).unwrap();
27     let mut vfs: VFS = VFS::new_with_stdio(StdioSys::new(
28         stdin: "a_string".as_bytes(),
29         stdout: std::fs::File::create(path: "./stdout").unwrap(),
30         std::io::stderr(),
31     ));
32
33     {
34         let mut memory_sys: WasiVirtualSys<MemoryDir, ...> = env::vfs::virtual_sys::WasiVirtualSys::<
35             env::vfs::impls::MemoryDir,
36             env::vfs::impls::MemoryFile,
37         >::new();
38         let tmp1: usize = memory_sys.create_dir_inode(dir_ino: 0, path: &"tmp").unwrap();
39         memory_sys WasiVirtualSys<MemoryDir, ...>
40             .create_file(dir_ino: tmp1, path: &"tmp1.txt", new_file: b"File content".to_vec().into()) Result<usi
41             .unwrap();
42
43         vfs.mount_file_sys(path: "mem", file_sys: Box::new(memory_sys));
44     }
45     let wasi_ctx: WasiCtx = WasiCtx::create_with_vfs(vfs);
46     let mut wasi: AsyncWasiModule = AsyncWasiModule::create_from_wasi_context(wasi_ctx).unwrap();
47     let mut instances: HashMap<String, &mut (dyn AsyncInst + Send)> = HashMap::new();
48     instances.insert(wasi.name().to_string(), wasi.as_mut());
49     let store: Store<'_, dyn AsyncInst + Send> = Store::new(None, instances).unwrap();
50     let module: Module = Module::from_file(None, wasm_file).unwrap();
51     let mut vm: Vm<'_, dyn AsyncInst + Send> = Vm::new(store);
52     vm.register_module(None, module).unwrap();
53     let _ = vm.run_func(None, "_start", []).await.unwrap();
54
55     println!("wasi exit_code = {}", wasi.exit_code())
56 } fn main
57
```

```
stdout
1 Ok("File content")
2
```

```
main.rs
1 fn main() {
2     let f = std::fs::read_to_string("mem/tmp/tmp1.txt");
3     println!("{}", f);
4 }
5
```

```
home > csh > try_rust > wasi_sleep > src > main.rs > ...
1 fn main() {
2     let f = std::fs::read_to_string("mem/tmp/tmp1.txt");
3     println!("{}", f);
4 }
5
```

```
问题 1 输出 调试控制台 终端 端口 1
→ wasmedgex git:(master) X cargo run ~/try_rust/wasi_sleep/target/wasm32-wasi/debug/wasi_sleep.wasm
wasi exit_code = 0
○ → wasmedgex git:(master) X
```



Part 03

使用 WASM 开发 Web 后端

问题背景

- **挑战：** 部署简单服务
- **典型流程：** 租用云服务器、环境配置、上传代码
- **需求：** 快速创建一个 简单的 Web 服务

解决方案（serverless）

- 无需管理服务器，专注于逻辑。
- 按需付费。
- 提高服务器资源利用率。

Flows 平台

- 基于 WASM 实现的 Serverless 平台。
- 快速灵活，无需复杂环境配置

GitHub review bot

 **Merged** support rust async && support https client #107
L-jasmine merged 11 commits into `main` from `feat/support_rust_async` 2 days ago

Hello, I am a [code review bot](#) on [flows.network](#). Here are my reviews of code commits in this PR.

Overall Summary:

The pull request includes several key changes, such as support for Rust async and support for HTTPS client. There are potential issues and errors that need to be addressed.

In the first pull request, there are potential problems related to the renaming of the "socket" function, modification of the Cargo.toml file, and updates to the examples and modules. The changes to multiple files and the commented-out code in the examples should also be reviewed.

In the second pull request, potential problems arise from the bumped version of libquickjs.a and the removal of constants in the binding.rs file. More context and explanations for these changes would be helpful.

The third pull request introduces potential problems with changes to multiple files, the large number of insertions and deletions, and the introduction of a new file "src/quickjs_sys/js_promise.rs". The updated and added dependencies should be reviewed, along with the purpose of the new "aho-corasick" dependency.

The fourth pull request raises concerns about the absolute path in the .cargo/config.toml file, the use of "println!" statements for debugging purposes, and the presence of commented-out sections in the code.

In the fifth pull request, issues arise from the use of "println!" statements, modifications to the fread and fwrite functions, and the lack of support for Rust async or HTTPS client as mentioned in the title.

Finally, the sixth pull request sees potential problems due to the absence of tests for the new HTTPS functionality, changes to multiple files, the removal of src/event_loop/poll.rs, changes to the Cargo.toml and Cargo.lock files, and conditional changes related to the "tls" feature.

Overall, there are common themes among the pull requests, such as the lack of sufficient explanations, potential issues with dependencies, and the need for proper testing. It would be beneficial to address these issues and provide more context to improve the review process and ensure the quality of the code changes.

Details

Commit 7547923419e957a8a5ad0b2f6207b1f467d0252f

Key changes:

Commit 41c92700ee5450d5f1f0f53cea1ae700d986732c

Key changes:

- Added `println!` statements for debugging purposes.
- Changed the target in the `.cargo/config.toml` file from `wasm32-wasi` to `/home/csh/WasmEdge/build/tools/wasmedge/wasmedge`
`--dir=...`

Potential problems:

- The `.cargo/config.toml` file changes the target to a specific path. It is unusual to include an absolute path in the configuration file. This might make the code harder to run or use in different environments.
- The addition of `println!` statements is helpful for debugging, but they should not be included in a production-ready codebase. It's important to remove or comment out these statements before merging the code.
- There are commented-out sections in the code, such as the `main` function and parts of the `async_run_with_context` implementation. These sections should be reviewed to ensure they are not relevant to the changes being made.

Overall, the patch contains changes that are mainly related to debugging and target configuration. It would be beneficial to address the potential problems mentioned above before merging the code.

Commit ebad9e7487507f9d7e05676772db98c46c265afc

Key Changes:

- In `src/internal_module/core.rs`, the `println!` statements have been commented out.
- In `src/internal_module/fs.rs`, the `fread` and `fwrite` functions have been modified to run asynchronously using the event loop.
- The calls to `println!` in the `fread` function have been added.

Potential Problems:

- The `println!` statements in `src/internal_module/fs.rs` should not be present in the production code. They should be removed or replaced with proper logging statements.
- The use of `println!` statements for debugging purposes is a potential source of performance issues and slowdowns. These statements should not be present in the production code.
- The modified `fread` and `fwrite` functions introduce asynchronous behavior, which may introduce new bugs or race conditions. These changes should be thoroughly tested to ensure correctness.
- The code does not appear to have any new support for Rust async or HTTPS client as mentioned in the PR title.

Learn rust

Michael Yuan 2023/09/12 12:51

@Learn Rust Can you tell me what is wrong with this code?

```
let mut text = "";
if text.is_empty() {
    let new_text = String::from("hello");
    text = &new_text;
}
call_function(text);
```

Learn Rust 机器人 2023/09/12 12:51

The code won't compile because of a mismatched data type and an issue with references and ownership.

In the first line, you are declaring `text` as a string slice (`&str`). However, in the following `if` statement, you're trying to assign `text` a reference to `new_text`, which is a `String` object.

Here's the corrected code:

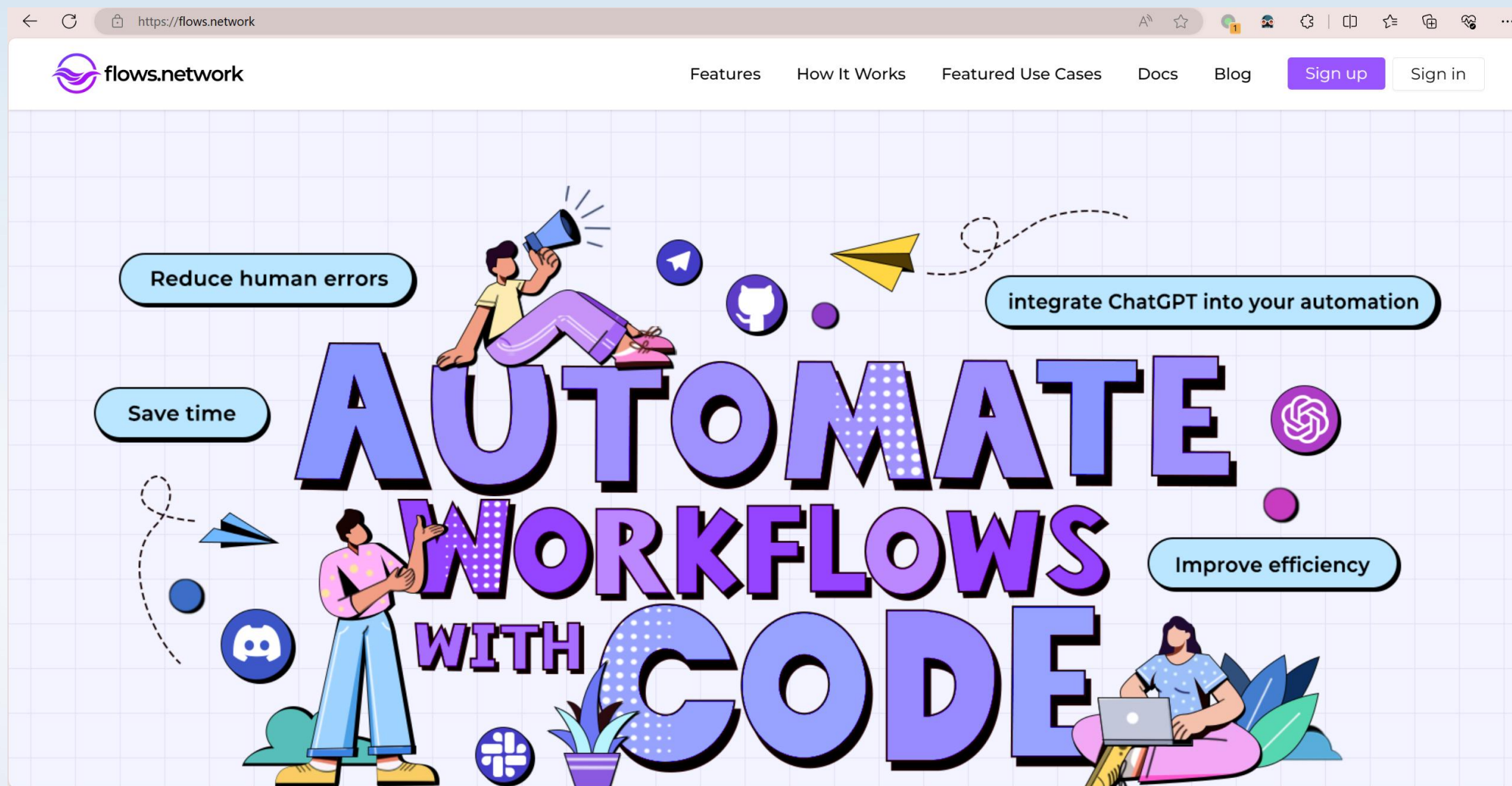
```
let mut text: String = String::new();
if text.is_empty() {
    let new_text = String::from("hello");
    text = new_text;
}
call_function(&text);
```

Or, you can also eliminate the unnecessary `new_text` variable:

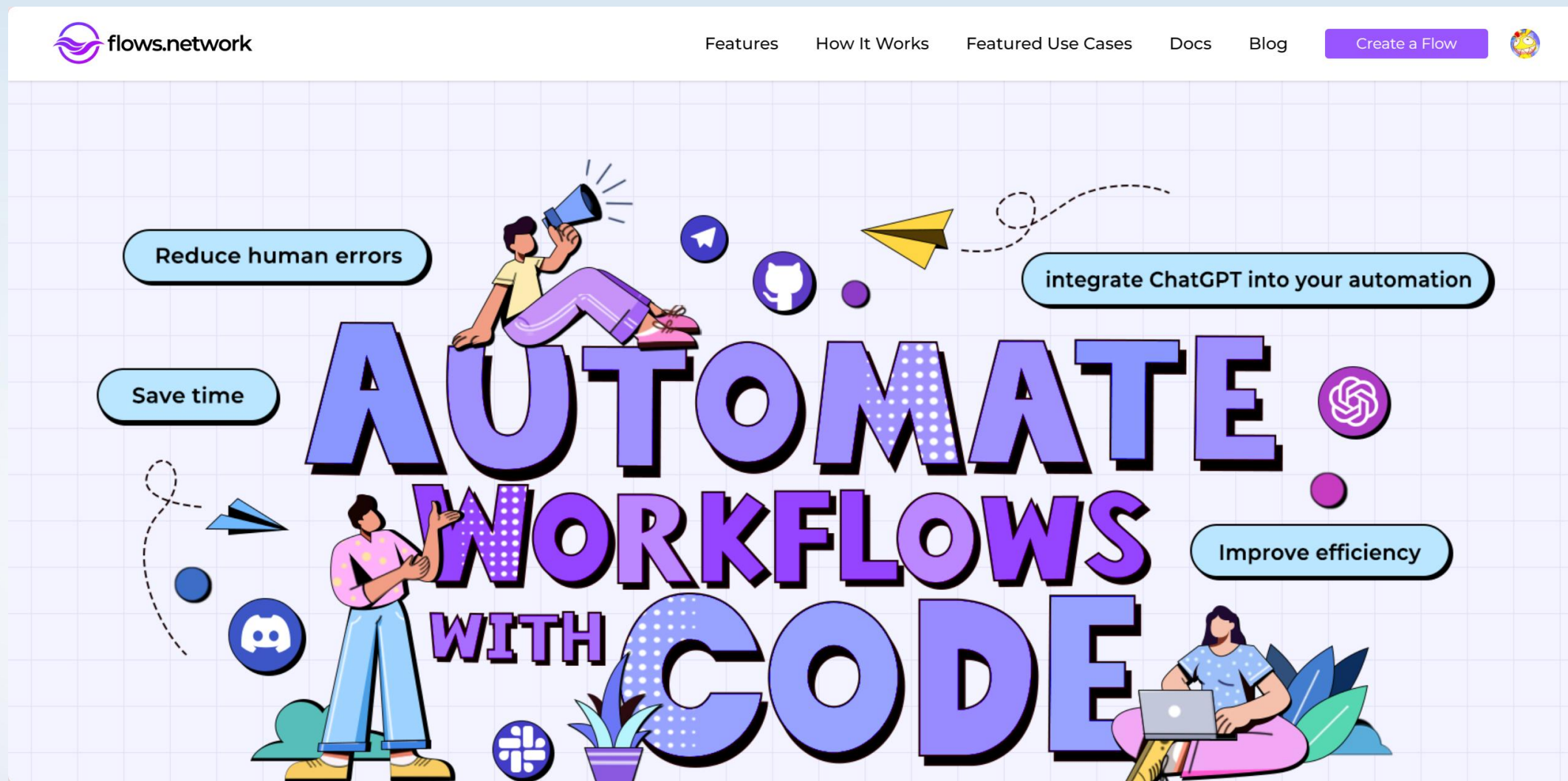
```
let mut text: String = String::new();
if text.is_empty() {
    text = String::from("hello");
}
call_function(&text);
```

In this corrected code, `text` is now of type `String`, the type returned from `String::from()`. We also make `text` mutable with `let mut`, so its value can be reassigned. Finally, when calling `call_function()`, we pass a reference to `text` using `&text`.

一个简单的 webhook



一个简单的 webhook



一个简单的 webhook



Change to your own prompts, language settings, or even use your own fine-tuned models to customize the bot's behavior.



Telegram

[>> Create from template](#)

Automate DevRel workflows for Open Source communities

Turn tedious manual workflows into automated flow functions. You can customize each flow based on your own requirements and approaches.



Discord notifications for new Hacker News articles with your keywords, plus a summary provided by ChatGPT.

[>> Create from template](#)



Slack notifications for new Hacker News articles with your keywords, plus a summary provided by ChatGPT.

[>> Create from template](#)



Slack notifications for negative sentiment GitHub issues By ChatGPT.

[>> Create from template](#)



Slack notifications for new Hacker News articles with your keywords.

[>> Create from template](#)



Slack notification for every 10 GitHub stars

[>> Create from template](#)



Slack notification for GitHub issues that hasn't been responded to in a certain number of days

[>> Create from template](#)



Discord notification for the GitHub issues with specified labels

[>> Create from template](#)



Slack notification for each fork of a GitHub repo

[>> Create from template](#)

Your use case is not covered by the templates? Create your own flow functions using our SDKs!

[Import & deploy the GitHub repo containing your flow source code](#)

一个简单的 webhook

< Create a Flow



Let's build a flow!

Import the flow function from a GitHub repo

Please select the GitHub repo that contains your flow function (i.e., the Rust source code of your bot). If you forked a flow function (bot) from a template, this should be your forked repo (eg my_name/my_forked_flow_func).

For example: If you are creating a GitHub automation bot, do NOT select the GitHub repo you want to deploy the bot here — you will select that later in the Integration step.

When you push code changes to the repo in the future, the flow function will be automatically rebuilt and redeployed.

Organization



L-jasmine



Repository

hello-world



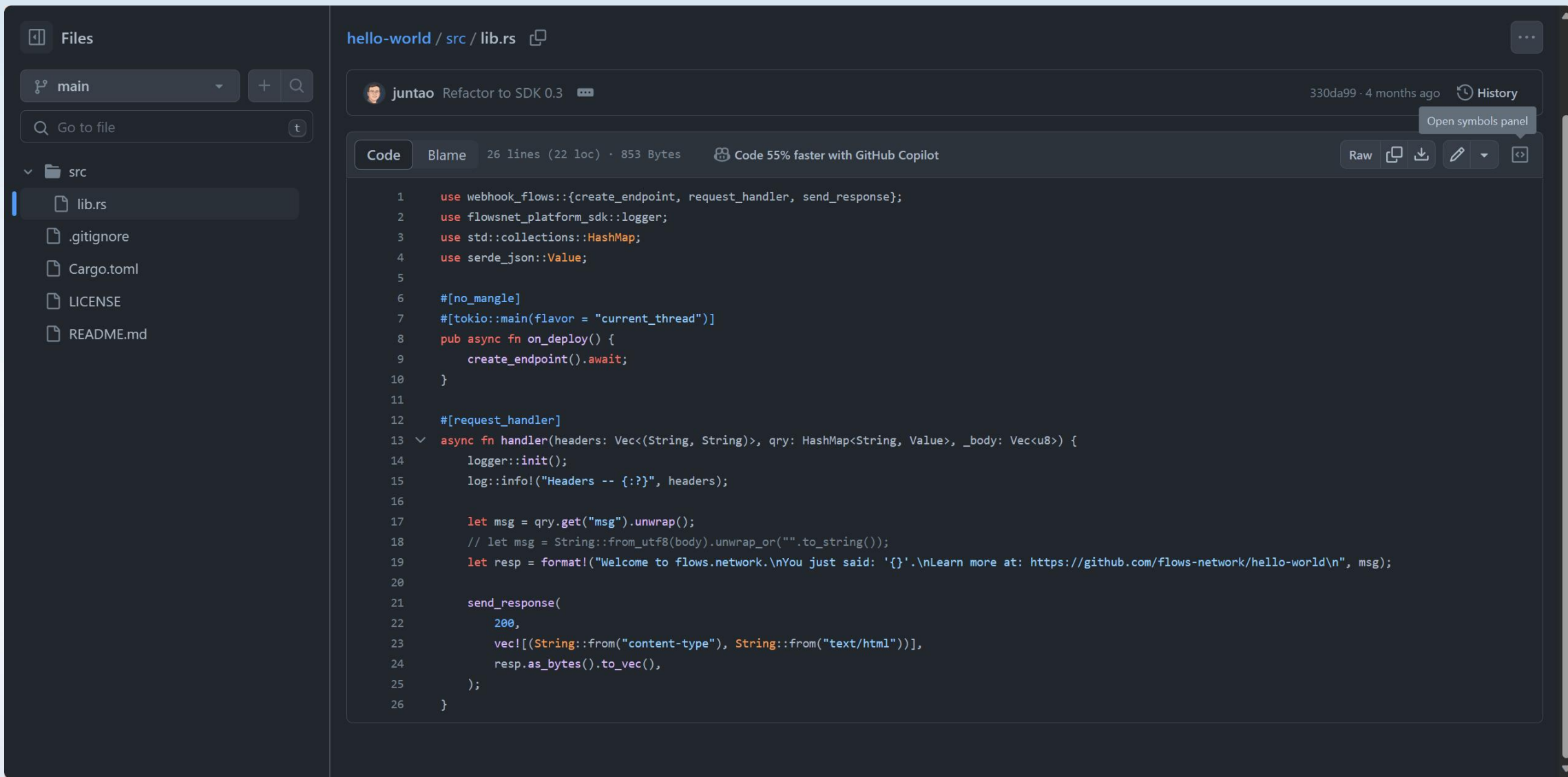
[Use a template >>](#)

Advanced

Build

*The default branch is the main branch and the directory is the root directory. If you need to customize your git repository settings, click on 'Advanced'.

一个简单的 webhook



Files

main

Go to file

src

lib.rs

.gitignore

Cargo.toml

LICENSE

README.md

hello-world / src / lib.rs

juntao Refactor to SDK 0.3 330da99 · 4 months ago History

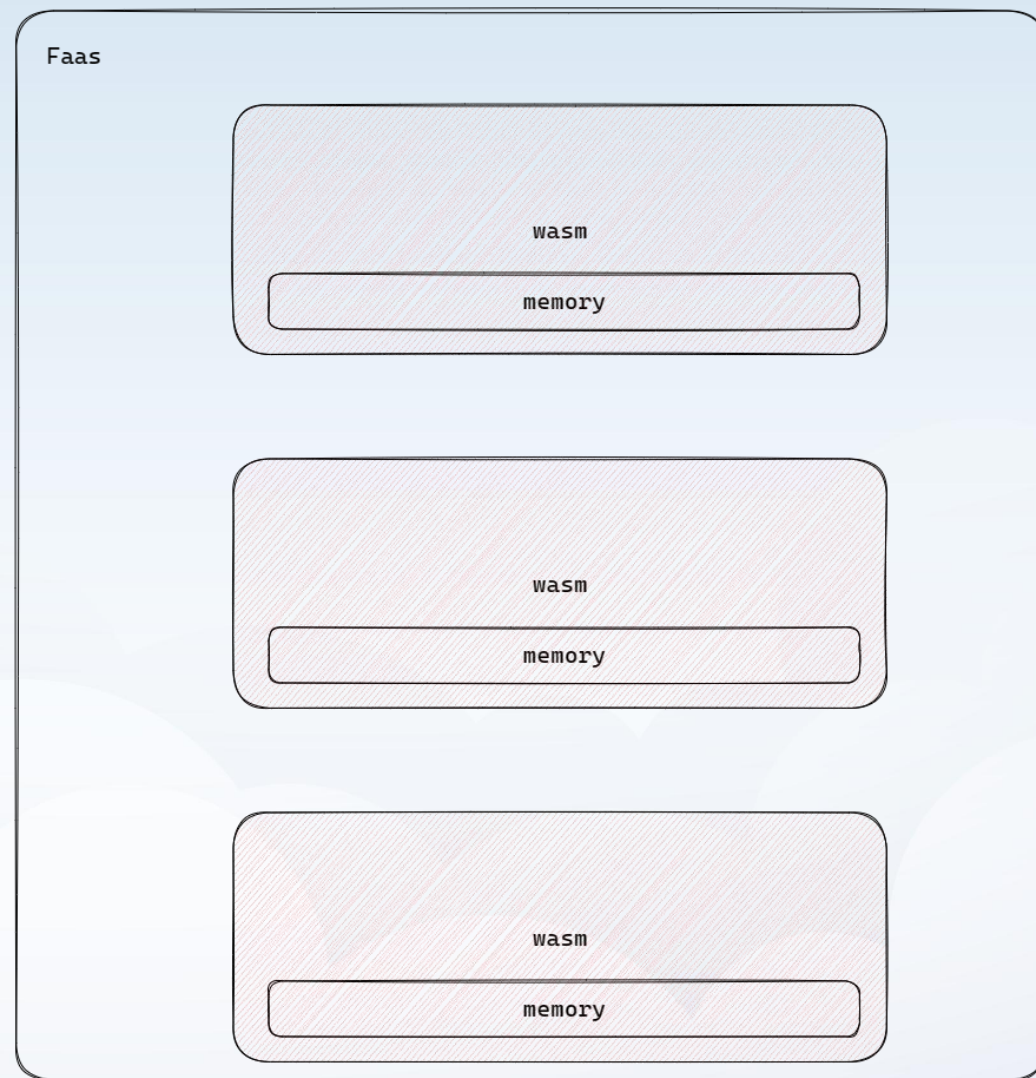
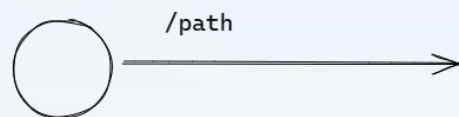
Open symbols panel

Code Blame 26 lines (22 loc) · 853 Bytes Code 55% faster with GitHub Copilot

Raw Copy Download Edit

```
1 use webhook_flows::{create_endpoint, request_handler, send_response};
2 use flowsnet_platform_sdk::logger;
3 use std::collections::HashMap;
4 use serde_json::Value;
5
6 #[no_mangle]
7 #[tokio::main(flavor = "current_thread")]
8 pub async fn on_deploy() {
9     create_endpoint().await;
10 }
11
12 #[request_handler]
13 async fn handler(headers: Vec<String, String>, qry: HashMap<String, Value>, _body: Vec<u8>) {
14     logger::init();
15     log::info!("Headers -- {:?}", headers);
16
17     let msg = qry.get("msg").unwrap();
18     // let msg = String::from_utf8(body).unwrap_or("").to_string();
19     let resp = format!("Welcome to flows.network.\nYou just said: '{}'.\nLearn more at: https://github.com/flows-network/hello-world\n", msg);
20
21     send_response(
22         200,
23         vec![(String::from("content-type"), String::from("text/html"))],
24         resp.as_bytes().to_vec(),
25     );
26 }
```

Serverless 的持久化问题



简单的 Todo

```
lib.rs x
src > lib.rs > update_tasks
8   };
9
10  use mysql_async::{prelude::*, Conn, Opts, OptsBuilder, Result, SslOpts};
11
12  #[no_mangle]
13  #[tokio::main(flavor = "current_thread")]
14  > pub async fn on_deploy() { ...
15
16
17
18  #[request_handler]
19  > async fn handler() { ...
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49  > fn get_db_url() -> String { ...
50
51
52
53
54
55
56
57  > async fn get_conn() -> Result<Conn> { ...
58
59
60
61
62
63
64
65
66  #[derive(Debug, serde::Deserialize, serde::Serialize)]
67  3 implementations
68  struct Task {
69      #[serde(default)]
70      id: u32,
71      description: String,
72      completed: bool,
73  }
74
75
76
77
78
79
80
81
82
83
84
85  > async fn add_tasks(_headers: Vec<(String, String)>, _qry: HashMap<String, Value>, body: Vec<u8>) { ...
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100 > async fn update_tasks(_headers: Vec<(String, String)>, qry: HashMap<String, Value>, body: Vec<u8>) { ...
101
102
103
104
105
106
107
108
109  async fn delete_tasks(
110      _headers: Vec<(String, String)>,
111      qry: HashMap<String, Value>,
112      _body: Vec<u8>,
113  ) { ...
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138  > async fn query(_headers: Vec<(String, String)>, _qry: HashMap<String, Value>, _body: Vec<u8>) { ...
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
```

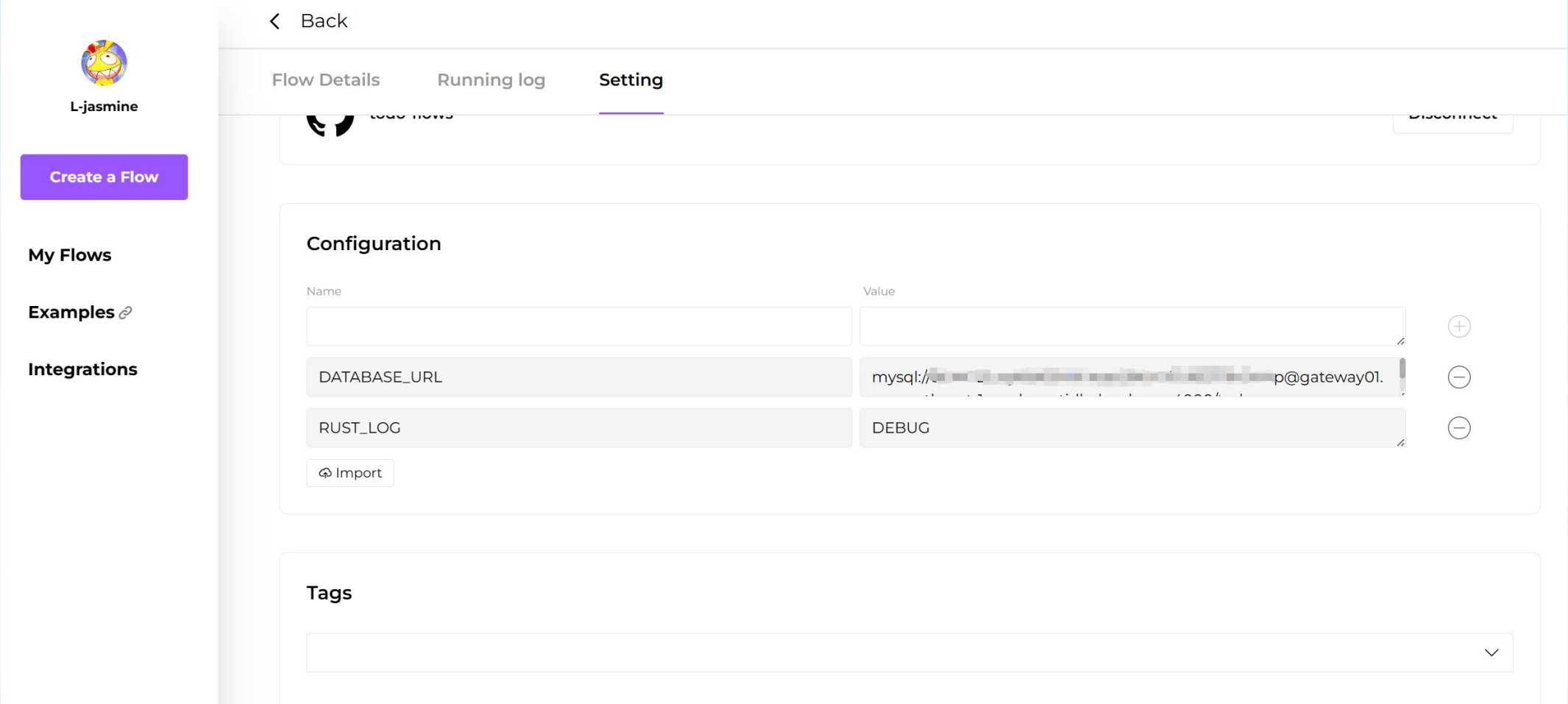
简单的 Todo

```
18 #[request_handler]
19 async fn handler() {
20     flowsnet_platform_sdk::logger::init();
21     let mut router = Router::new();
22     router
23         .insert("/tasks", vec![get(query), post(add_tasks)])
24         .unwrap();
25     router
26         .insert("/tasks/:id", vec![put(update_tasks), delete(delete_tasks)])
27         .unwrap();
28
29     if let Err(e) = route(router).await {
30         match e {
31             RouteError::NotFound => {
32                 send_response(
33                     404,
34                     vec![],
35                     serde_json::to_vec(&serde_json::json!({"err": "No route matched"})).unwrap(),
36                 );
37             }
38             RouteError::MethodNotAllowed => {
39                 send_response(
40                     405,
41                     vec![],
42                     serde_json::to_vec(&serde_json::json!({"err": "Method not allowed"})).unwrap(),
43                 );
44             }
45         }
46     }
47 }
```

简单的 Todo

```
49 fn get_db_url() -> String {
50     if let Ok(url: String) = std::env::var(key: "DATABASE_URL") {
51         url
52     } else {
53         "mysql://root:pass@127.0.0.1:3306/mysql".into()
54     }
55 }
56 |
57 async fn get_conn() -> Result<Conn> {
58     let db_url: String = get_db_url();
59     log::debug!("connect db {db_url}");
60     let opts: Opts = Opts::from_url(&db_url)?;
61     let builder: OptsBuilder = OptsBuilder::from_opts(opts);
62     let builder: OptsBuilder = builder.ssl_opts(SslOpts::default());
63     Conn::new(opts: builder).await
64 }
65
```

简单的 Todo



简单的 Todo

```
85  async fn add_tasks(_headers: Vec<(String, String)>, _qry: HashMap<String, Value>, body: Vec<u8>) {
86      let mut conn: Conn = get_conn().await.unwrap();
87
88      let task: Task = serde_json::from_slice(&body).unwrap();
89
90      match r"insert into tasks (description,completed) values (:description,:completed)" &str
91          .with(params! {"description"=>&task.description,"completed"=>&task.completed}) QueryWithParams<&str, Params>
92          .ignore(&mut conn) Pin<Box<dyn Future<Output = ...> + Send>>
93          .await
94      {
95          Ok(_) => ok(&task),
96          Err(e: Error) => ok(&serde_json::json!({"err":e.to_string()})),
97      }
98  }
99
100  async fn update_tasks(_headers: Vec<(String, String)>, qry: HashMap<String, Value>, body: Vec<u8>) {
101      log::debug!("qry={qry:?}");
102      let id: u32 = qry.get("id").unwrap().as_str().unwrap().parse().unwrap();
103
104      let mut task: Task = serde_json::from_slice(&body).unwrap();
105
106      task.id = id;
107      let mut conn: Conn = get_conn().await.unwrap();
108
109      match r"update tasks set description=:description,completed=:completed where id = :id" &str
110          .with(params! {"id"=>id,"description"=>&task.description,"completed"=>&task.completed}) QueryWithParams<&str, Params>
111          .ignore(&mut conn) Pin<Box<dyn Future<Output = ...> + Send>>
112          .await
113      {
114          Ok(_) => ok(&task),
115          Err(e: Error) => ok(&serde_json::json!({"err":e.to_string()})),
116      }
117  }
118
```


简单的 Todo



L-jasmine

Create a Flow

My Flows

Examples [🔗](#)

Integrations

Sign Out

[Back](#)

Flow Details **Running log** [Setting](#)

todo-flows ● Running

This flow has been called 65 times in this month.

Imported GitHub Repo



Owner/Repo/Branch: [L-jasmine/todo-flows](#)

Status: ● Ready

Deployment Log [rebuild](#)

>  Build

Webhook Endpoint

[Copy](#)

```
~ 设置 + - x
- ~ curl -X GET https://code.flows.network/webhook/MFEC0a7P2SrCzKxXq9JH/tasks
[{"id":1,"description":"learn rust","completed":false}]%
- ~ curl -X POST https://code.flows.network/webhook/MFEC0a7P2SrCzKxXq9JH/tasks -d '{"descri
ption":"learn rust","completed":false}'
{"id":0,"description":"learn rust","completed":false}%
- ~ curl -X GET https://code.flows.network/webhook/MFEC0a7P2SrCzKxXq9JH/tasks
[{"id":1,"description":"learn rust","completed":false}, {"id":2,"description":"learn rust","c
ompleted":false}]%
- ~ curl -X DELETE https://code.flows.network/webhook/MFEC0a7P2SrCzKxXq9JH/tasks/1
{"id":1}%
- ~ curl -X GET https://code.flows.network/webhook/MFEC0a7P2SrCzKxXq9JH/tasks
[{"id":2,"description":"learn rust","completed":false}]%
- ~
```


适用场景

Web 服务

- **Serverless 平台：**根据请求的内容，动态加载和执行相应的WASM模块来处理请求。

数据处理场景

- **自定义二进制协议解析：**解析和处理自定义的二进制数据协议。

规则触发场景

- **事件驱动：**在特定事件或规则触发时，执行用户定义的动作或逻辑。
- **规则描述：**允许用户制定并执行针对特定数据的分析规则。

共性总结

- **高度定制化：**难以统一抽象描述的用户定制化需求。
- **频次高：**频繁且随机的启动和执行。
- **持续时间短：**不像普通的web服务，需要一直持续运行。



Thanks.

